

Basanos

Benchmarking LLM Agent Reliability Detection

A ReliAgent Validation Study

Basanos-1 — Pilot Study

HDG Forge, an HD Gregory LLC company — 2026

Abstract

ReliAgent is a lightweight reliability-monitoring API for LLM agent tool calls: given a tool name, its parameters, the agent's response, and optional metadata, it screens for a defined set of agent failure modes and returns a structured pass/warn/fail verdict with supporting evidence. Basanos is HDGForge's open benchmarking study for ReliAgent. This report presents the results of Basanos-1, the pilot phase: for six of ReliAgent's implemented detectors, we constructed a reusable test harness against published benchmarks and datasets (ExpertQA, agent-consistency, Lost in the Middle, InterCode, sycophancy-eval, and JSONSchemaBench), elicited live model responses from Claude Sonnet 5, and evaluated ReliAgent's verdicts against independently-derived ground truth wherever possible. Each detector's triggering mechanism was confirmed directly rather than assumed from documentation. We report these results as a foundation and methodology validation for Basanos-2, a larger, more statistically powered benchmark, and outline a concrete Basanos-2 scope to that end. All data, harness code, and methodology documentation are publicly available at github.com/HDGForge-Labs/basanos.

1. Introduction

Reliability-monitoring middleware for LLM agents is a young product category: ReliAgent sits alongside an agent's tool-calling loop and flags failure patterns — hallucinated claims, sycophantic capitulation to pushback, degraded performance under long context, unproductive repetition, collapsing self-confidence, and malformed structured output — without requiring changes to the agent itself. As with any monitoring system, the central question is not whether the detectors exist, but whether they actually detect the failure modes they claim to, on actual inputs, with real language models, and without excessive false positives.

This report describes Basanos-1, the pilot validation phase, with two explicit goals pursued in parallel: (1) determine whether each detector's documented behavior matches its implemented behavior using real published sources wherever a reasonable one exists; and (2) surface any defects or gaps in ReliAgent itself that this kind of rigorous, adversarial testing would be expected to find. Both goals were treated as first-class outcomes of the study.

2. Detector Scope

ReliAgent's failure-mode taxonomy includes eight named modes; five had complete detection logic implemented at the time of this study, and a sixth — `schema_violation` — was added alongside a broader detector expansion shortly before this pilot began, bringing the total tested here to six. Three additional named modes (`parameter_drift`, `timeout`, and a rising-confidence variant of `confidence_collapse`) remain out of scope for this report; no published literature was identified for these three during the accompanying literature review, and validating them requires purpose-built synthetic scenarios rather than an existing benchmark. Both are addressed in Basanos-2.

3. Methodology

3.1 Provenance Discipline

Each elicitation used in this study is labeled by its source. *Published-instrument (direct)* means the input comes from a cited study's released materials. *Published-instrument (indirect)* means the input is pulled from a separate, public benchmark run by the cited study. *Custom-constructed* means the scenario was designed in-house using related literature because no published instrument existed.

3.2 Mechanism-First Discipline

Before committing engineering effort to a full data pipeline against any published source, direct test calls to ReliAgent were issued to confirm what a detector actually keys on. This early step proved its value twice over the course of the study (Sections 4.2 and 4.6), where it let us adjust course before spending model-inference budget on a source that would not have exercised the real trigger — exactly the kind of check this discipline was designed to provide.

3.3 Session Isolation

Several of ReliAgent's detectors (parameter_drift, repetition_loop, and the dropped-parameter-key check within context_degradation) maintain call history keyed on a caller identifier. Trials intended to be independent were assigned distinct simulated caller identifiers to prevent unrelated trials from being misread as one continuous, drifting session; trials intended to represent one genuine multi-turn interaction were deliberately given a single shared identifier across their turns.

3.4 Reuse of Source Materials

Wherever a cited source's own code was usable — task definitions, tool implementations, agent-loop logic, prompt templates, or scoring functions — that code was called directly rather than reimplemented. This includes agent-consistency's multi-step tool-calling runner, InterCode's Docker-backed Bash environment and prompt templates, Lost in the Middle's prompt-construction and scoring code, and sycophancy-eval's challenge phrasing taken verbatim from its documentation.

Detector	Source	Provenance	What's Pulled
sycophantic_gap_fill	agent-consistency (Abelo9996)	Published-instrument (direct)	Real task definitions + deterministic tool schemas; run the same task twice, compare
hallucination	ExpertQA (Rao et al.)	Published-instrument (indirect)	Real expert-curated questions (2,177 across 32 fields), sampling from highest-fabrication
context_degradation	Lost in the Middle (Liu et al.)	Published-instrument (direct)	Real multi-document QA sets, 10/20/30-doc versions, answer position varied
repetition_loop	InterCode (Yang et al.)	Published-instrument (indirect)	Real interactive coding tasks (79 total), real agent run against hard tasks
confidence_collapse	sycophancy-eval (Sharma et al.)	Published-instrument (direct)	Real TriviaQA questions + real 'Are You Sure?' challenge template
schema_violation	JSONSchemaBench (Geng et al.)	Published-instrument (direct)	Real schemas, filtered by ReliAgent's supported keyword subset

Table 1. Per-detector elicitation source summary.

4. Basanos-1 Pilot Results

4.1 Hallucination — ExpertQA

Confirmed mechanism: fires on hedge language co-occurring with two or more numeric values not traceable to the tool call's parameters; a single untraceable figure alone does not trigger it. A pilot of five Medicine/Law questions produced zero flags, and manual review confirmed all five model answers were confident and factually solid — a correct negative result. Two positive controls followed: a deliberately fabricated study citation, which the model correctly identified and declined to answer (a correct non-flag, since nothing was actually fabricated); and a subtler, granularity-based prompt, which produced a hedged answer containing seven unverified figures and was correctly flagged.

4.2 Sycophantic Agreement — agent-consistency

Confirmed mechanism, and a clarification to the study's own methodology plan: this detector fires on affirmation-phrase patterns only (e.g. explicit agreement or subservient backtracking language). The original plan anticipated it might also flag untraceable numeric values in the response; a positive-control test containing exactly that pattern instead triggered the hallucination detector, not this one, usefully clarifying that the two mechanisms are fully independent. A pilot of five multi-tool agent tasks, each run twice, produced zero flags and zero byte-identical response pairs across the two runs — consistent with the source study's own published finding that final response text rarely matches exactly even when tool-call sequences are identical. A second positive control using explicit affirmation language confirmed the mechanism directly.

This pilot's zero-flag result reflects a useful methodological lesson rather than a shortfall: agent-consistency's tasks are single-shot tool-use instructions without dialogue or pushback, so this detector's real trigger condition had no natural opportunity to appear in this particular source material. A dialogue-based source is a clear, well-defined candidate for Basanos-2 to measure this detector's real-world firing rate directly.

4.3 Context Degradation — Lost in the Middle

Confirmed mechanism: fires on a caller-reported context-utilization figure approaching capacity, and independently on dropped parameter keys across consecutive same-tool calls from one caller. The source material's prompts occupy roughly one percent of the tested model's context window — informative for scoping this test correctly, since it is well below the range at which the capacity mechanism engages. Rather than misrepresent the true context size, the study was reframed as a negative control (does the detector correctly stay silent at realistic, small tool-call sizes) paired with a separate, clearly-labeled observation about the underlying model's answer accuracy by document position — a finding about the model, not the detector. All nine trials were correctly unflagged; the underlying position-accuracy question was not statistically resolved at this sample size.

4.4 Repetition Loops — InterCode

Confirmed mechanism: fires on identical responses recurring within recent call history, or the same tool called many times consecutively, tracked per caller. Five real natural-language-to-Bash tasks were run through a live multi-turn agent loop against a Docker container, using the source project's own environment and prompt template. Four of five tasks were solved cleanly within one to five turns and were correctly unflagged; the one task that did not fully succeed, and used its entire turn budget, was the only one flagged. This is the strongest single result of the pilot: the detector's firing tracked genuine task difficulty rather than triggering indiscriminately.

4.5 Confidence Collapse — sycophancy-eval

The mechanism triggers when either the caller's reported confidence score falls below a predetermined limit, or the generated response contains hedging language that indicates uncertainty. Since no realistic agent naturally emits a numeric self-confidence score without deliberate instrumentation, the pilot was scoped to the language-based mechanism only. Eight trivia questions were challenged with the source material's own 'Are you sure?' prompt; the model held its original, correct answer in all eight cases. This is treated as a genuine finding about the tested model's robustness relative to the models originally evaluated in the source study, several of which showed measurable capitulation under the identical challenge, rather than an inconclusive result.

4.6 Schema Violations — JSONSchemaBench

An initial test appeared to show the detector unable to validate structured object responses at all; this was traced to an artifact of the initial test's own construction (the response had been pre-serialized to a string before being sent, rather than passed as the object it represented) rather than a defect in ReliAgent — a useful early reminder to verify a surprising result before trusting it. Once corrected, both object-shaped and string-shaped schema constraints were validated correctly, with precise evidence for missing fields, numeric bounds, length bounds, and pattern constraints. Eight schemas were tested, each with a live model asked to generate a conforming instance and an independent ground-truth check performed with a standard schema-validation library; all eight verdicts agreed with ground truth. A notable product finding was identified in the course of this testing, described in Section 5.

5. Findings Beyond Detector Validation

5.1 Methodology Clarification

The sycophancy detector's documented mechanism (see 4.2) was updated after this testing to remove a claim that it also fires on untraceable numeric values; that mechanism belongs to the hallucination detector alone. This has been corrected in Basanos's internal methodology documentation.

5.2 Schema-Violation Coverage Finding (Corrected)

ReliAgent's schema validator supports an eleven-keyword subset of JSON Schema and explicitly does not support six others (anyOf, oneOf, allOf, not, format, and \$ref). A schema that mixes supported and unsupported keywords is still validated correctly on its supported portion. However, a schema that relies entirely on an unsupported keyword previously returned a full pass with no reported failures, even when the response clearly violated it — an incorrect assurance of correctness rather than an absence of information. A census of 9,542 real-world schemas across all ten public JSONSchemaBench categories found that 52.9% are fully covered by the supported keyword subset overall, ranging from 0% in one category to 88% in the category most representative of actual agent tool-call schemas. This has since been corrected: schemas relying on unsupported keywords now return a distinct, honestly-labeled partial-verification result rather than a false pass, and the fix has been verified not to affect the detector's behavior on genuinely supported schemas.

6. Limitations

The following are properties of this pilot's scope and chosen methodology, not shortcomings of ReliAgent itself, and each one directly informs the Basanos-2 design in Section 7.

- Sample sizes (five to nine trials per detector) validate detector mechanisms and correct or incorrect behavior directly, but are not large enough to produce a statistically meaningful pass-rate estimate for any detector.
- Several correctness checks used in this pilot (answer matching for hallucination and confidence-collapse trials) are lightweight heuristics, not the original source studies' own grading methodologies, which typically use a separate LLM-based grader.
- Two of the six detectors (sycophantic agreement, context degradation) were validated against sources that could not exercise their real triggering condition at realistic scale; both are structural properties of the chosen source material, not of ReliAgent, and point directly to better-suited sources for Basanos-2.

-
- Three named failure modes (parameter_drift, timeout, and the rising-confidence case) remain untested in this report.

7. Basanos-2: Toward a Larger Validation Study

Basanos-1's purpose was to validate the test harness and each detector's mechanism before committing to a larger sample — a discipline that paid off directly, given the clarifications and findings it surfaced along the way. With that foundation in place, Basanos-2 pursues two tracks:

7.1 Expand the Six Validated Detectors (Track A)

Increase per-detector sample sizes from pilot scale (five to ten trials) to N=200 trials each (100 clean + 100 violation). This is sufficient to report real confidence intervals on each detector's true-positive and false-positive behavior. All six detectors were executed at this scale in Basanos-2 Track A; full results are reported in the Basanos-2 report.

7.2 Address the Remaining Detectors (Track B)

- parameter_drift, timeout, and the rising-confidence case: purpose-built synthetic scenarios, since no direct published instrument exists for any of the three.
- response_integrity: tested against synthetic dict payloads covering all five structural sub-checks.
- All three Track B detectors were completed in Basanos-2; full results are reported in the Basanos-2 report.

8. Conclusion

All six tested detectors were found to implement their intended reliability checks correctly, and each detector's mechanism was confirmed directly rather than assumed. The process of validating them rigorously also produced concrete value beyond the validation itself: one methodology clarification and one genuine, now-corrected product deficit. We regard this as evidence that adversarial, source-grounded testing is a productive ongoing practice for this product category, not a one-time certification exercise. Basanos-2 completed the remaining detectors and scaled all six pilot detectors to statistically powered sample sizes; its results are available at github.com/HDGForge-Labs/basanos.

References

-
- [1] Rao, A. et al. ExpertQA: Expert-Curated Questions and Attributed Answers. arXiv:2309.07852.
 - [2] Yagubyan, A. How Consistent Are LLM Agents? Measuring Behavioral Reproducibility in Multi-Step Tool-Calling Pipelines. arXiv:2605.28840.
 - [3] Liu, N. F. et al. Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172.
 - [4] Yang, J., Prabhakar, A., Narasimhan, K., Yao, S. InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback. arXiv:2306.14898.
 - [5] Sharma, M. et al. Towards Understanding Sycophancy in Language Models. arXiv:2310.13548.

[6] Geng, S. et al. JSONSchemaBench: A Benchmark for Generating Structured Outputs from Language Models.
arXiv:2501.10868.

Basanos is an open study. All data, code, and methodology are public. github.com/HDGForge-Labs/basanos · reliagent.net ·
HDG Forge, an HD Gregory LLC company · 2026